

Agent 101

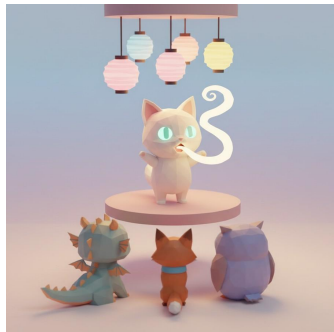
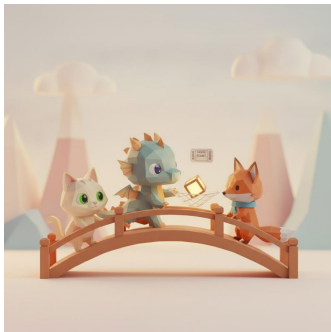
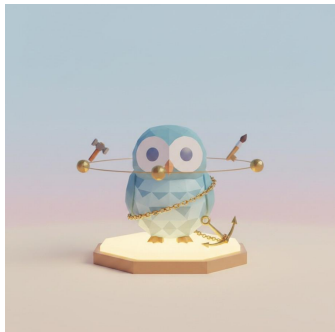
Week 1



One agent tonight. A system by week five.

Each week starts with the problem the week before hands you.

► TONIGHT



WEEK 1

Control

A model can talk, but it cannot do anything.

one agent you can trust

WEEK 2

Decompose

One agent cannot hold a whole job.

many that don't collide

WEEK 3

Coordinate

Things fail halfway, and money moves twice.

flows that survive failure

WEEK 4

Remember

It forgets who you are between sessions.

memory you designed

WEEK 5

Live

Reading is not watching, or listening.

sees and speaks, live

goo.gle/agent101-ep1-credit

GCP Credit



goo.gle/agent101-lab1

Hands on Lab



Agenda

1 Your First Agent

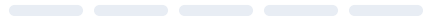
2 Tools

3 State

4 Callback

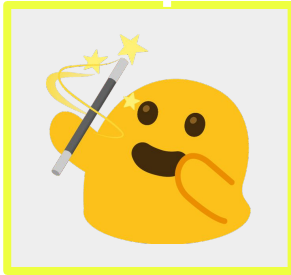
PART ONE

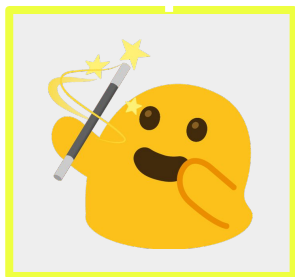
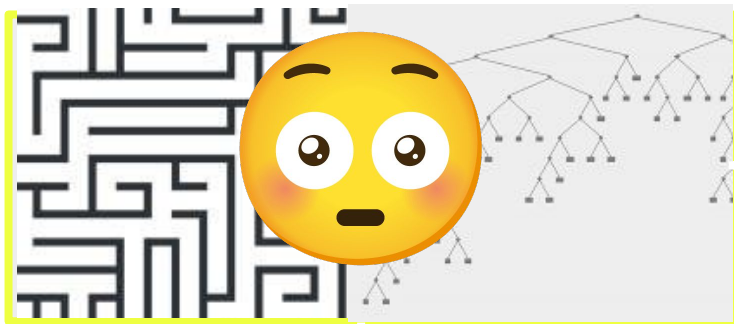
Why Agent



Familiar Summoner

Traditional
Application

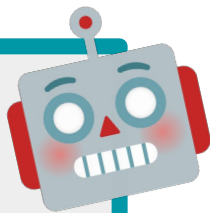




Agent

System Prompt

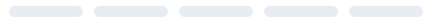
State, Memory



PART ONE

What is an agent

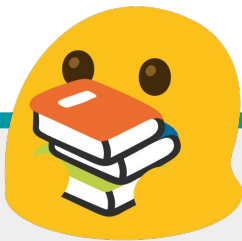
Two things you already know how to build, and
the one they add up to.



Agent

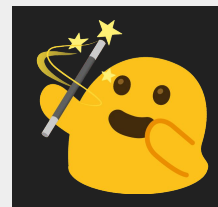
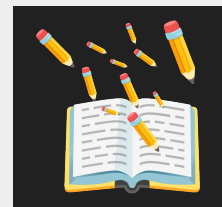
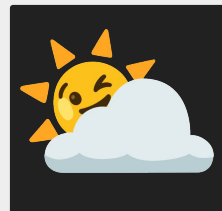
“You are a commute planner...”

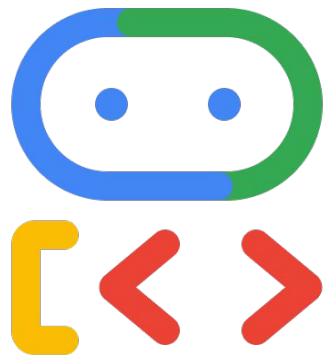
session ID 78234331



Model

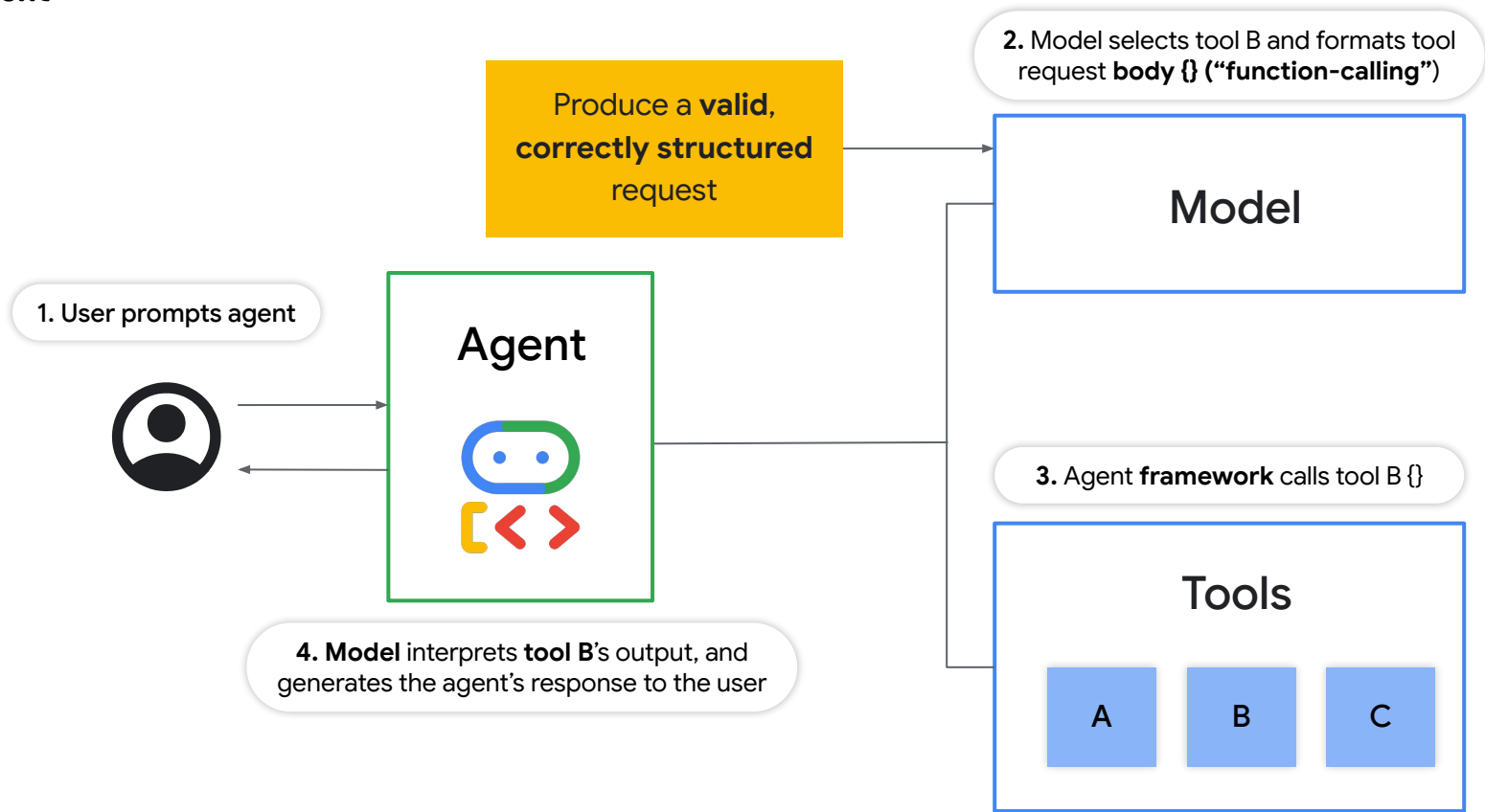
Tools





**Build, evaluate, and
deploy sophisticated AI
agents with code-first
flexibility!**

Agent

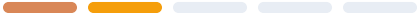


```
root_agent = LlmAgent(  
    name="grove",  
    model="gemini-3-flash-preview",  
    instruction=INSTRUCTION,  
)
```

PART FOUR

A tool

One name. One thing it can now actually do.



```
root_agent = LlmAgent(  
    name="grove",  
    model="gemini-3-flash-preview",  
    instruction=INSTRUCTION,  
    tools=[cast_candidates],  
)
```

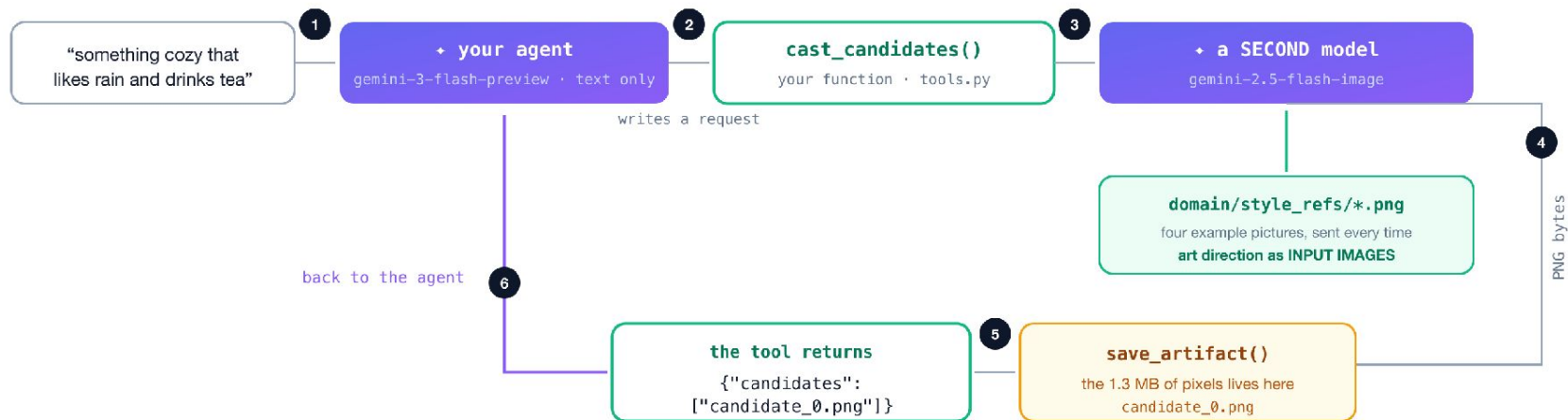

ALL THE MODEL EVER READS

```
async def cast_candidates(description: str):  
    """Draw the familiar a traveler described.  
  
    Args:  
        description: species, colours, mood.  
    """
```

Write it for a reader, not a compiler.

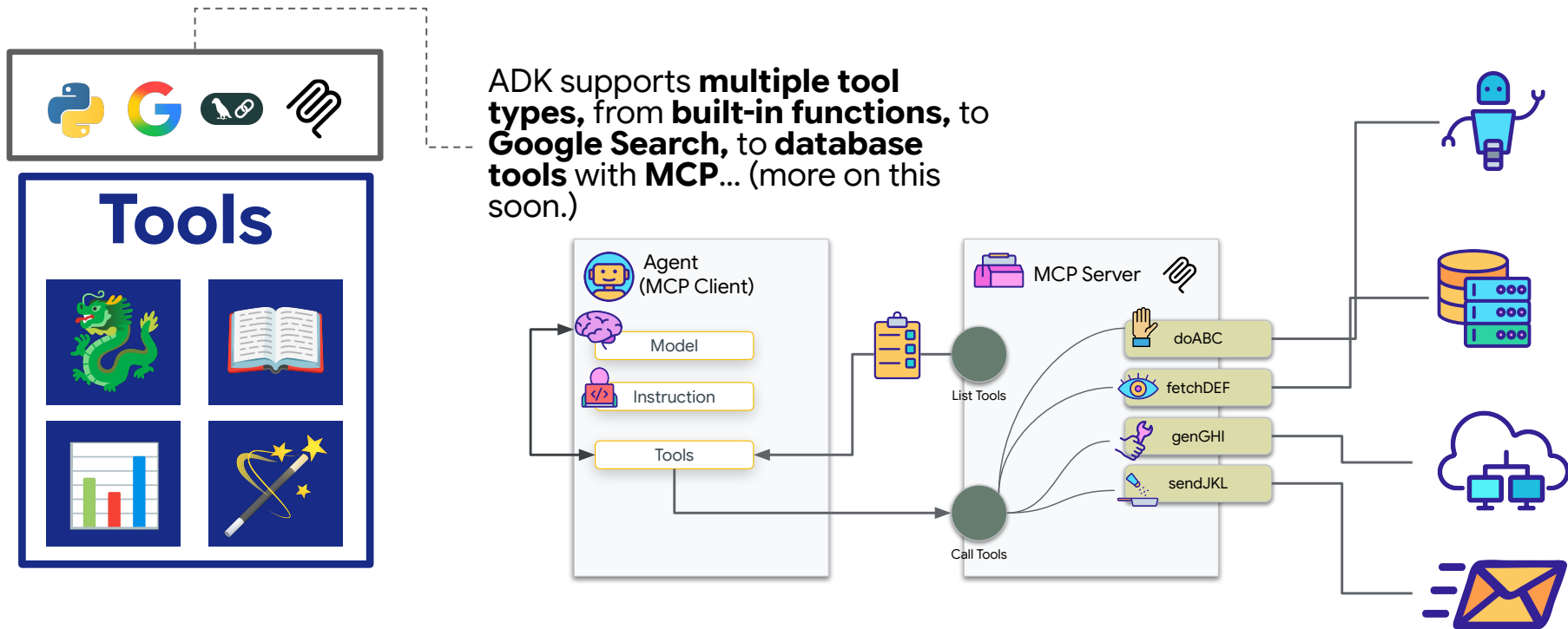
Where a picture actually comes from

Your agent is a text model. It never draws anything — it asks a function to, and that function calls a second model.



Tools and MCP

An agent **tool** is a **capability** that helps an agent **interact with the world**.
(An **API**, a **database**, another **agent**...)



PART FIVE

State

Remember that line in the panel? This is where it came from.



How the tool talks back

They never call each other. They share one dictionary — the tool writes into it, the agent's prompt reads out of it.



The tool wrote it. The prompt read it. Neither one imported the other.

PART SIX

The reference

Four outfits, one face. Nobody re-typed a description.



Same face, new outfit

canon



written down once

every render after



anchored back to canon

PART SEVEN

A callback

Every turn builds on the one before it. Someone has to decide on what.



Call backs



Manage State:

Read or dynamically update the agent's session state



before agent call back

Implement Guardrails

Enforce safety rules, validate inputs/outputs, or prevent disallowed operations



Base Agent

Before model callback

Model

after model callback

Before tools callback

Tools

After tools callback



after agent call back



Caching:

Temporary data store



Observe & Debug:

Log detailed information at critical steps for monitoring and troubleshooting



Notification:

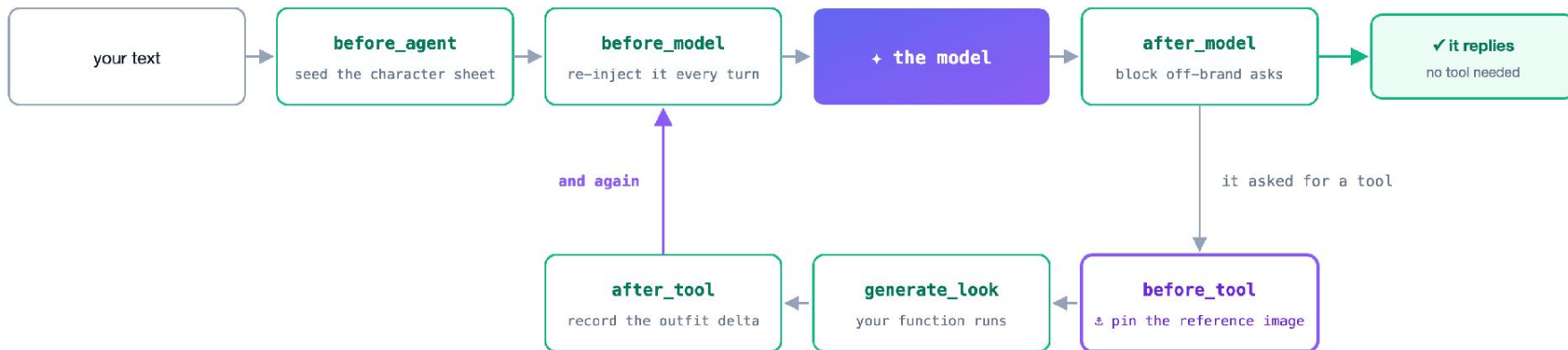
Tracing, or notify human

Control:

Modify data flowing through the agent or bypass certain steps.

Five places you can stand

A callback is your function, running at a named moment inside the turn. Same turn as before — now with doors in it.



before_tool is the one this lab makes you feel.

It runs between the model asking and your function running — the only place you can still change the arguments. Here it decides WHICH image the next render is anchored to: the familiar you claimed (📌 pinned), or whatever came out last time (💎 unpinned, and it slowly stops being your familiar).

The instruction is a request. A callback is a rule.

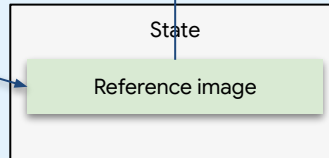
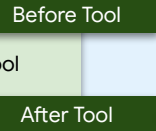
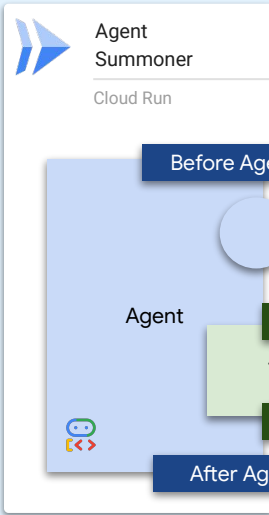
Anything you cannot afford to have the model forget does not belong in the prompt — it belongs in a callback.

Callback

Callback

Orchestration

Agent



AI



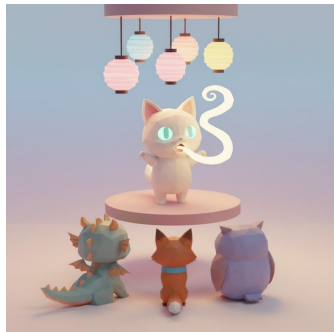
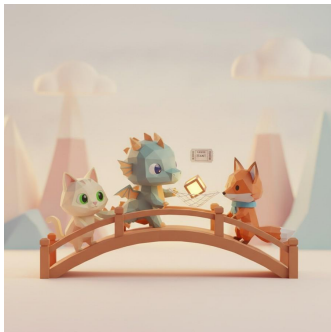
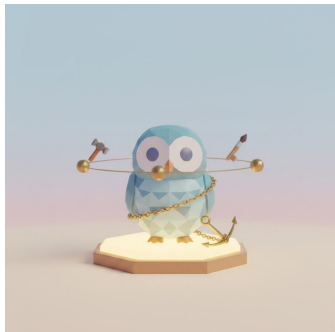
Gemini 3.0
Flash

VertexAI

One agent tonight. A system by week five.

Each week starts with the problem the week before hands you.

► TONIGHT



WEEK 1

Control

A model can talk, but it cannot do anything.

one agent you can trust

WEEK 2

Decompose

One agent cannot hold a whole job.

many that don't collide

WEEK 3

Coordinate

Things fail halfway, and money moves twice.

flows that survive failure

WEEK 4

Remember

It forgets who you are between sessions.

memory you designed

WEEK 5

Live

Reading is not watching, or listening.

sees and speaks, live

Thank you!

— —

